

Intro to DH

EGR 445/545

GVSU

Plan for Today

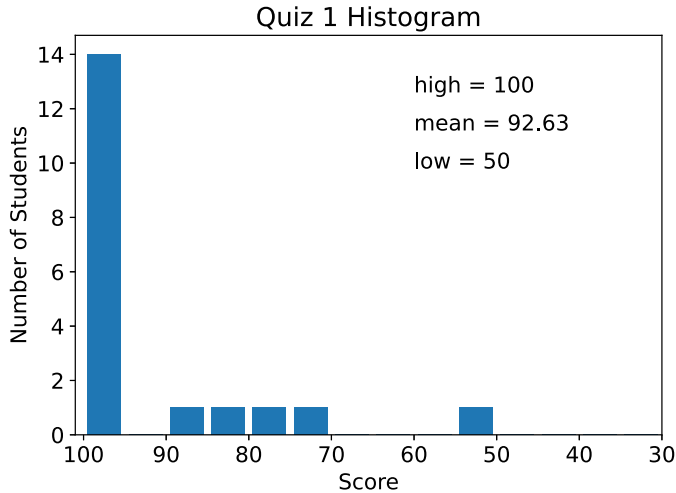
- quiz 1 solution
- wrap-up chapter 2
- intro to DH (chapter 3)

Announcements

- HW 2 and LA 7 are up

Quiz 1 Solution

Quiz 1 Histogram



Quiz 1 Solution

Chapter 2 Wrap-Up

What are Euler angles and why should we care?

- someday you may have to talk to an old, grumpy engineer
- do rotation matrices really have 9 unknowns?

$$\mathbf{R} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

$${}^A_R = \begin{bmatrix} {}^A\hat{x}_B & {}^A\hat{y}_B & {}^A\hat{z}_B \end{bmatrix}$$

- what constraints are placed on a valid rotation matrix?
 - and where do those constraints come from?

$$\begin{aligned} \hat{x} \cdot \hat{x} &= 1 & \hat{x} \cdot \hat{y} &= 0 & \hat{x} \cdot \hat{z} &= 0 & \hat{z} \cdot \hat{z} &= 1 \end{aligned}$$

orthonormal

$$\hat{y} \cdot \hat{y} = 1 \quad \hat{y} \cdot \hat{z} = 0$$

Euler Angles: What do we mean by “arbitrary rotation”?

- are rotation matrices restricted to x , y , or z axes?
- what use would we have for an arbitrary rotation matrix?

Euler Angles

- **big ideas:**

- an *arbitrary* rotation matrix has three unknowns that can be represented using three angles
- an *arbitrary* rotation/orientation can be expressed as the product of three rotation matrices:

$${}^A_B\mathbf{R} = \mathbf{R}(\alpha) \cdot \mathbf{R}(\beta) \cdot \mathbf{R}(\gamma)$$

- we primarily focus on rotations about a single axis
- Euler angles handle the more general case of arbitrary orientation between frames A and B

Free Vectors

What are free vectors and how to we deal with them?

- are some vectors not free?
 - is this a social justice issue?
- is there a whale or dolphin that needs help getting back to the ocean?
- how much do regular vectors cost?
 - do we get free stuff?

Free vs. Line Vectors

- a free vector can be placed anywhere
- the opposite of a free vector is a line vector
- the line of action is important for a line vector
 - you cannot move a line vector without affecting its meaning
- positions and forces are examples of line vectors
- velocities and moments are examples of free vectors

Velocities in different frames

- imagine an AGV in an Amazon warehouse
- the AGV is being observed by two different stations
 - station {B} is rotated and translated from station {A}
 - the velocity of the AGV would have different components in the two frames
 - should the magnitude of the velocity be different in the two frames?

Alternative Velocity Question

$${}^A\mathbf{P} = {}^A_B\mathbf{R} {}^B\mathbf{P} + {}^A\mathbf{P}_{BORG}$$

- velocity is the derivative of position
- taking derivatives in moving reference frames is graduate dynamics
- if ${}^A\mathbf{P}_{BORG}$ is constant, what happens when we take the derivative?

Computational considerations

- what is the fastest way to compute ${}^0\mathbf{P}$?

$${}^0\mathbf{P} = {}^0_1\mathbf{T}_1 {}^1_2\mathbf{T}_2 {}^2_3\mathbf{T}_3 \mathbf{P}$$

Computational considerations: which is faster?

- option 1: Find ${}^0_3\mathbf{T}$ first

$${}^0_3\mathbf{T} = {}^0_1\mathbf{T} {}^1_2\mathbf{T} {}^2_3\mathbf{T}$$
$${}^0\mathbf{P} = {}^0_3\mathbf{T} {}^3\mathbf{P}$$

Handwritten annotations: 4×4 (green), 4×4 (blue), 4×1 (purple).

Option 1 has more calculations

- option 2: Find ${}^2\mathbf{P}$, then find ${}^1\mathbf{P}$, and finally ${}^0\mathbf{P}$

$${}^2\mathbf{P} = {}^2_3\mathbf{T} {}^3\mathbf{P}$$
$${}^1\mathbf{P} = {}^1_2\mathbf{T} {}^2\mathbf{P}$$
$${}^0\mathbf{P} = {}^0_1\mathbf{T} {}^1\mathbf{P}$$

Handwritten annotations: 4×1 (purple) for each equation.

Intro to DH

Connections

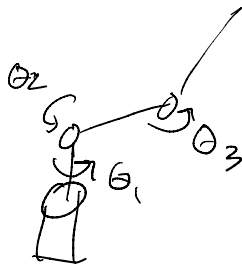
joint angles $\leftrightarrow$ position and orientation of frame N :

- what is the fundamental challenge in industrial robotics?
- connections: you have learned rotation and HT matrices in chapter 2
 - how does this help us in chapter 3?
 - are HT matrices alone enough for industrial robot kinematics?

0T_N

DH Intro: Plexiglas Robot

- do you know enough to model the kinematics of the Plexiglas robot using HT matrices?



Why do we need a standard procedure?

- if each of you chose your own approach to attaching the frames and defining the HT matrices, what could go wrong?
 - we have already seen some of this
 - what things have kept us from all getting to the same solution using the same steps?
 - is it important that we use the same intermediate steps?

→ which axes do I rotate about?
→ where do the origins go?

Big Ideas for Chapter 3

- any robot can be modeled with a series of HT matrices:

$${}^0P_{tip} = {}^0_1\mathbf{T} \cdot {}^1_2\mathbf{T} \cdot {}^2_3\mathbf{T} \cdot {}^3_4\mathbf{T} \cdot {}^4_5\mathbf{T} \cdot {}^5_6\mathbf{T} \cdot {}^6P_{tip}$$

- any robot link can be modeled using an HT matrix according to eqn. 3.6
 - two rotations and two translations
 - first, rotation and translation involving $\hat{X}_{i-1}$
 - then rotation and translation involving $\hat{Z}_i$
 - essentially, $HT_x \cdot HT_z$

Eqn. 3.6 in a nut shell

Any robot link can be modeled as the product of two HT matrices, one involving X_{i-1} and the other involving Z_i :

$${}^{i-1}_i\mathbf{T} = HT_x(\alpha_{i-1}, x = a_{i-1}) \cdot HT_z(\theta_i, z = d_i)$$

$${}^{i-1}_i\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & a_{i-1} \\ 0 & \cos(\alpha_{i-1}) & -\sin(\alpha_{i-1}) & 0 \\ 0 & \sin(\alpha_{i-1}) & \cos(\alpha_{i-1}) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) & 0 & 0 \\ \sin(\theta_i) & \cos(\theta_i) & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Key Points for Chapter 3

- Figure 3.5/3.15
- Derivation of eqn. 3.6
- DH parameter definitions
- DH Frame attachment procedure

Key Parts of Chapter 3 - restated

- If you understand how **eqn. 3.6** is derived based on **Figure 3.15** and you understand the **parameter definitions** and **frame attachment** summaries from **section 3.4**, the rest of chapter 3 falls into place.

DH Frame Attachment Procedure

Attachment Procedure - Book

page 74, section 3.4

Summary of link-frame attachment procedure

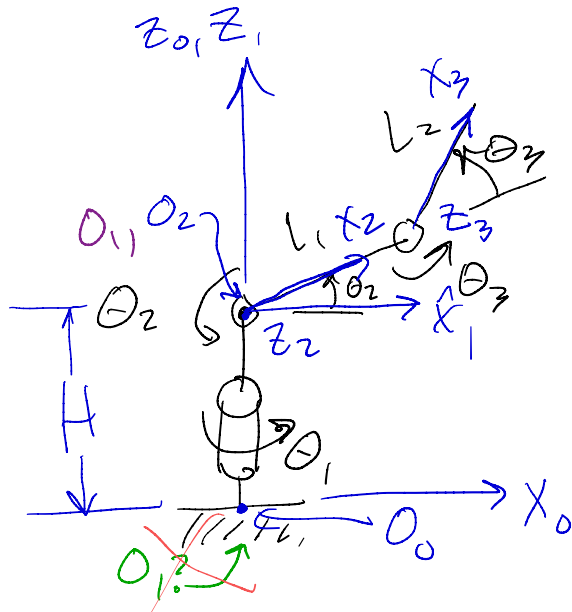
The following is a summary of the procedure to follow when faced with a new mechanism, in order to properly attach the link frames:

1. Identify the joint axes and imagine (or draw) infinite lines along them. For steps 2 through 5 below, consider two of these neighboring lines (at axes i and $i + 1$).
2. Identify the common perpendicular between them, or point of intersection. At the point of intersection, or at the point where the common perpendicular meets the i th axis, assign the link-frame origin.
3. Assign the $\hat{Z}_i$ axis pointing along the i th joint axis.
4. Assign the $\hat{X}_i$ axis pointing along the common perpendicular, or, if the axes intersect, assign $\hat{X}_i$ to be normal to the plane containing the two axes.
5. Assign the $\hat{Y}_i$ axis to complete a right-hand coordinate system.
6. Assign $\{0\}$ to match $\{1\}$ when the first joint variable is zero. For $\{N\}$, choose an origin location and $\hat{X}_N$ direction freely, but generally so as to cause as many linkage parameters as possible to become zero.

DH Frame Attachment Summary

1. Identify the joint axes
2. Identify the common perpendiculars or the point of intersection between the joint axes
3. Assign the $\hat{Z}_i$ axes along the i^{th} joint axes
4. Assign the $\hat{X}_i$ axes along the mutual perpendiculars or perpendicular to both $\hat{Z}$ axes if two consecutive $\hat{Z}_i$ intersect
5. Assign the $\hat{Y}$ axes to complete a right-hand coordinate system
 - normally skipped
6. Decide on frames $\{0\}$ and $\{N\}$

DH Example: Plexiglas Robot



LA 1

θ_i is ALWAYS a rotation about $\hat{z}_i$
 → actuated joints rotate about $\hat{z}$ axes

egm. 36: $L=0, a=0, \theta=\theta, d=0$

$${}^0_1T = H T_z(\theta_1, 0, 0, 0)$$

LA1 Pleriglas Robot DH table

i	α_{i-1}	q_{i-1}	θ_i	d_i
1	0	0	θ_1	H
2	$+90^\circ$	0	θ_2	0
3	0	L_1	θ_3	0

- to get from frame 1 to frame 2, I first rotate by α_1 about $\hat{x}_1$, translate by a_1 along $\hat{x}_1$
- then I rotate by θ_2 about $\hat{z}_2$ and translate by d_2 along $\hat{z}_2$

If two Z axes intersect, then
the lower numbered origin MUST
be placed at the intersection